Спецификация модуля ККД: тестовое задание системного аналитика

Спецификация описывает модель данных, правила проверки, метрики, расчёт дельты и контракт API модуля контроля качества данных по целям: по ней разработчик реализует модуль без возврата к аналитику. Она опирается на документ бизнес-аналитика и использует те же правила, метрики и числа.

Связь с документом бизнес-аналитика

Спецификация закрывает вопросы, которые документ бизнес-аналитика сознательно оставил системному аналитику, и воспроизводит его числа. Каждая строка ниже показывает, откуда требование взято и где оно описано.

Из документа бизнес-аналитика	Где закрыто в спецификации
Активная цель: «в текущий период ведётся работа, не черновик, не закрытая и не отменённая», точные статусы фиксирует системный аналитик	Раздел 2: статус `ACTIVE` и срок `deadline` не раньше даты снимка
Вопрос 2: цели на вершине иерархии не обязаны иметь вышестоящую цель	Раздел 2, правило `CASCADE_NO_PARENT`: проверяются только цели подразделений с вышестоящим подразделением
Вопрос 6: как сравнивать после сбоя	Раздел 4: прошлым считается ближайший успешный прогон
Четыре правила: «Цель без ответственного», «У цели нет ключевых результатов», «Веса KP не дают 100%», «Цель не связана с вышестоящей»	Раздел 2: те же названия, важность и рекомендации, плюс коды и условия на SQL
Три метрики и правило «нет данных вместо нуля»	Раздел 3: формулы и значения для пустых случаев (`null`)
История проблемы: сколько проверок подряд сохраняется	Раздел 5: поле `runs_streak` в списке проблем
Эскалация критичных проблем	Раздел 5: фильтры `severity` и `delta_status` в списке проблем

Из документа бизнес-аналитика	Где закрыто в спецификации
Сценарии приёмки 1–4	Раздел 5: пример ответа построен на числах сценария 1, остальные сценарии — граничные случаи

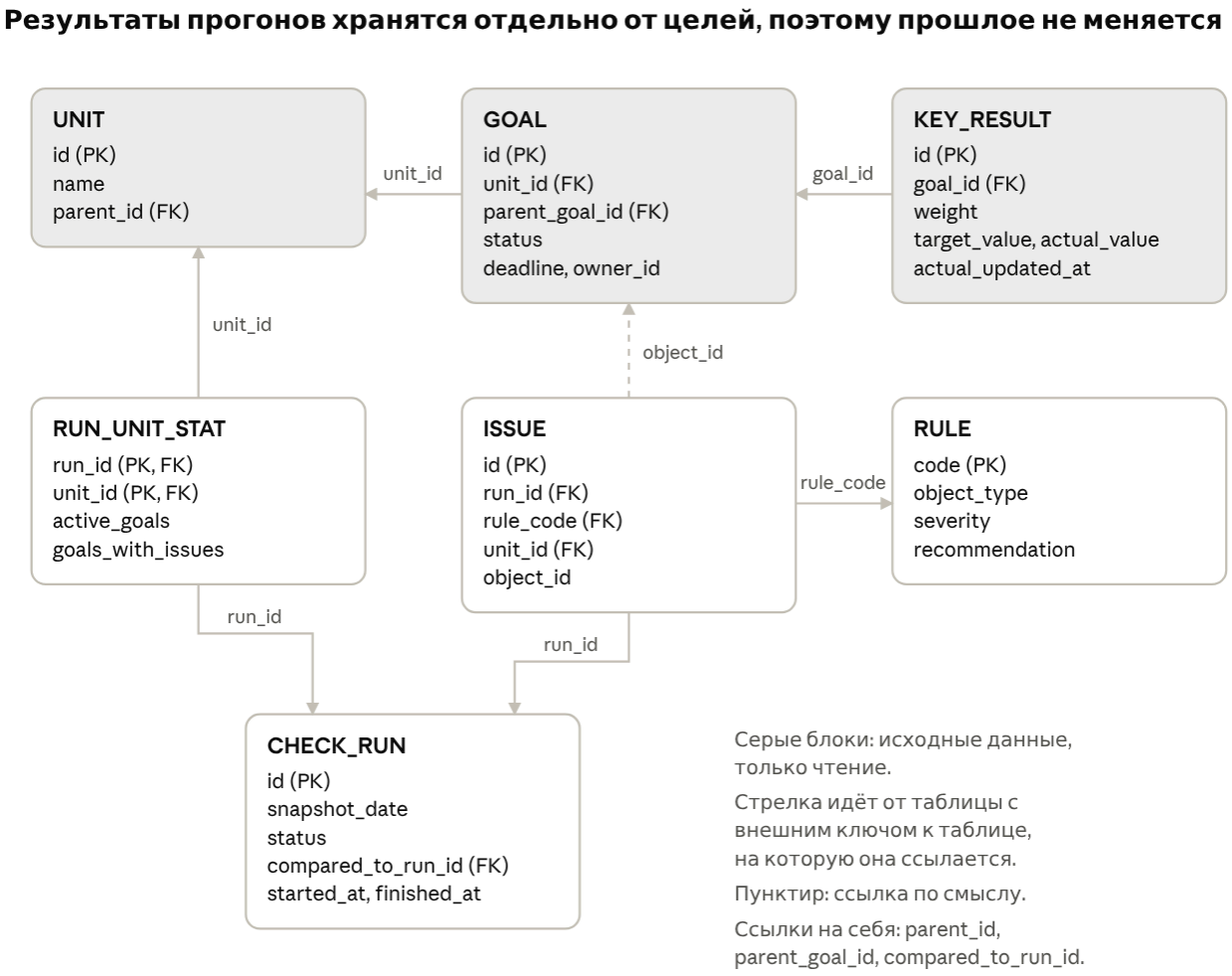
Соответствие терминов

Документ бизнес-аналитика	Спецификация
Проверка	Прогон (`check_run`)
«Нет данных»	`null`
Критично / существенно / незначительно	`critical` / `major` / `minor`
60%	`60.00`
Подразделение вместе со всеми дочерними	Охват подразделения: само и все дочерние

1. Модель данных

Модель состоит из трёх сущностей исходных данных системы управления целями (только чтение) и четырёх сущностей результатов ККД. Результаты каждого прогона фиксируются, поэтому прошлые прогоны не меняются, когда меняются цели.

ER-диаграмма



ER-диаграмма · 7 сущностей

Диаграмма показывает ключевые поля и связи, полный состав атрибутов приведён в таблицах ниже. Таблица `issue` также ссылается на `unit` через `unit_id`.

Исходные данные (только чтение)

`unit` — подразделение

Атрибут	Тип	Обязательность	Описание
`id`	varchar	да, первичный ключ	Идентификатор, например `HQ-CIB`
`name`	varchar	да	Название
`parent_id`	varchar	нет	Вышестоящее подразделение. Пусто у подразделения на вершине иерархии

`goal` — цель

Атрибут	Тип	Обязательность	Описание
`id`	varchar	да, первичный ключ	Идентификатор цели
`unit_id`	varchar	да	Ссылка на `unit`
`title`	varchar	да	Формулировка цели
`owner_id`	varchar	нет	Идентификатор ответственного из корпоративного справочника сотрудников. Пусто, если ответственный не назначен
`parent_goal_id`	varchar	нет	Вышестоящая цель. Пусто, если связи нет
`status`	enum	да	`DRAFT`, `ACTIVE`, `CLOSED`, `CANCELLED`
`deadline`	date	да	Срок достижения цели

`key_result` — ключевой результат

Атрибут	Тип	Обязательность	Описание
`id`	varchar	да, первичный ключ	Идентификатор КР
`goal_id`	varchar	да	Ссылка на `goal`
`title`	varchar	да	Формулировка КР
`target_value`	numeric	нет	Целевое значение
`actual_value`	numeric	нет	Фактическое значение

Атрибут	Тип	Обязательность	Описание
`weight`	numeric(5,2)	да	Вес KP в процентах, от 0 до 100
`actual_updated_at`	timestamp	нет	Дата обновления фактического значения

Результаты ККД (записываются модулем)

`rule` — справочник правил

Атрибут	Тип	Обязательность	Описание
`code`	varchar	да, первичный ключ	Код правила, например `GOAL_NO_OWNER`
`title`	varchar	да	Название для пользователя
`object_type`	enum	да	Объект проверки: `GOAL` или `KEY_RESULT`
`severity`	enum	да	`critical`, `major`, `minor`
`recommendation`	varchar	да	Рекомендация пользователю: как исправить

`check_run` — прогон

Атрибут	Тип	Обязательность	Описание
`id`	varchar	да, первичный ключ	Идентификатор, например `RUN-008`
`snapshot_date`	date	да	Дата снимка данных
`status`	enum	да	`RUNNING`, `SUCCESS`, `FAILED`
`compared_to_run_id`	varchar	нет	Прошлый прогон, с которым сравниваются результаты. Пусто у первого прогона
`started_at`	timestamp	да	Начало выполнения
`finished_at`	timestamp	нет	Завершение. Пусто, пока прогон выполняется

`issue` — найденная проблема

Атрибут	Тип	Обязательность	Описание
`id`	varchar	да, первичный ключ	Идентификатор записи проблемы
`run_id`	varchar	да	Ссылка на `check_run`
`rule_code`	varchar	да	Ссылка на `rule`
`object_id`	varchar	да	Идентификатор объекта, в котором найдена проблема (сейчас всегда цель)
`unit_id`	varchar	да	Подразделение объекта на момент прогона
`severity`	enum	да	Важность на момент прогона (копия из правила)
`details`	varchar	да	Что именно не так

Ограничение уникальности: `(run_id, rule_code, object_id)`.

`run_unit_stat` — счётчики подразделения по прогону

Атрибут	Тип	Обязательность	Описание
`run_id`	varchar	да	Ссылка на `check_run`
`unit_id`	varchar	да	Ссылка на `unit`
`active_goals`	integer	да	Число активных целей самого подразделения (без дочерних) на момент прогона
`goals_with_issues`	integer	да	Сколько из них имеют хотя бы одну проблему

Первичный ключ: `(run_id, unit_id)`. Подразделение без целей имеет запись с нулями.

Связи

- `unit` → `unit`: у подразделения не более одного вышестоящего (`parent_id`).
- `unit` 1→N `goal`; `goal` → `goal`: у цели не более одной вышестоящей цели (`parent_goal_id`); `goal` 1→N `key_result`.
- `check_run` → `check_run`: прогон сравнивается не более чем с одним прошлым прогоном.
- `check_run` 1→N `issue`; `check_run` 1→N `run_unit_stat`; `rule` 1→N `issue`.

Ключевые решения

Решение	Почему
Результаты и счётчики хранятся по прогонам	Цели меняются каждую неделю. Если считать прошлый прогон по текущим данным, цифры задним числом изменятся, и сравнение «было / стало» станет неверным
Отдельная таблица `run_unit_stat`	Знаменатель доли корректных целей («все активные цели на момент прогона») нигде, кроме снимка, не сохранён
Ответственный — внешний идентификатор	Справочник сотрудников ведётся в корпоративной системе, его копия в ККД не нужна

Решение	Почему
`weight` обязателен, от 0 до 100	Иначе правило про сумму весов пришлось бы описывать отдельно для пустых значений
`issue.severity` копируется из правила	Если важность правила изменят позже, прошлые прогоны не перепишутся
`issue.object_id` не является внешним ключом	В будущем проблема может относиться и к KP (`rule.object_type`), поэтому ссылка задана по смыслу

2. Правила проверки

Четыре правила проверяют активные цели и их KP. Каждое задано текстом и запросом на SQL. Если текст и SQL разойдутся, верным считается текст.

Границы набора. Задание для разработчиков предлагает шесть правил, спецификация формализует четыре. Правила `KR_NO_TARGET` и `KR_STALE_PROGRESS` не входят. Поля `target_value` и `actual_updated_at` остаются в модели как часть описания KP, уровень `minor` зарезервирован для будущих правил.

Что считается активной целью

- **Активная цель** — запись `goal` , у которой `status = 'ACTIVE'` и `deadline >= :snapshot_date` , где `:snapshot_date` — дата снимка прогона (`check_run.snapshot_date`).
- **Активный KP** — запись `key_result` , у которой `goal_id` указывает на активную цель.
- Цели со статусами `DRAFT` , `CLOSED` , `CANCELLED` не проверяются. Цель со статусом `ACTIVE` , срок которой истёк раньше даты снимка, тоже не проверяется. Цель со сроком ровно в день снимка проверяется.

Все правила читают данные из одного общего выражения, а снимок данных делается один раз при старте прогона:

```
WITH active_goal AS (
  SELECT * FROM goal
  WHERE status = 'ACTIVE' AND deadline >= :snapshot_date
)
```

Правило 1. `GOAL_NO_OWNER`

Поле	Значение
Название	Цель без ответственного
Объект проверки	цель (`GOAL`)
Важность	`critical`
Условие нарушения	У активной цели поле `owner_id` пусто: `NULL` или строка из пробелов
`issue.object_id`	`goal.id`
`issue.details`	`Ответственный не назначен`

```
SELECT g.id AS object_id, g.unit_id
FROM active_goal g
WHERE g.owner_id IS NULL OR btrim(g.owner_id) = '';
```

Правило 2. `GOAL_NO_KR`

Поле	Значение
Название	У цели нет ключевых результатов
Объект проверки	цель (`GOAL`)
Важность	`critical`
Условие нарушения	У активной цели нет ни одной записи `key_result` с `goal_id` этой цели
`issue.object_id`	`goal.id`
`issue.details`	`У цели нет ключевых результатов`

```
SELECT g.id AS object_id, g.unit_id
FROM active_goal g
WHERE NOT EXISTS (SELECT 1 FROM key_result k WHERE k.goal_id = g.id);
```

- По такой цели создаётся только эта проблема: правило `KR_WEIGHTS_SUM` к ней не применяется.

Правило 3. `KR_WEIGHTS_SUM`

Поле	Значение
Название	Веса КР не дают 100%
Объект проверки	цель (проверяется набор её КР)
Важность	`major`
Условие нарушения	У активной цели есть хотя бы один КР, и сумма `weight` всех её КР не равна 100
`issue.object_id`	`goal.id`
`issue.details`	`Сумма весов КР: {сумма}%, ожидается 100%`

```
SELECT g.id AS object_id, g.unit_id, SUM(k.weight) AS weights_sum
FROM active_goal g
JOIN key_result k ON k.goal_id = g.id
GROUP BY g.id, g.unit_id
HAVING SUM(k.weight) <> 100;
```

- Соединение `JOIN` отбрасывает цели без КР, поэтому цель без КР даёт только проблему `GOAL_NO_KR`, и правила не дублируют друг друга.
- Сравнение точное, без допуска. Тип `numeric(5,2)` хранит значения без погрешностей округления: веса 33,33 + 33,33 + 33,33 дают 99,99, это нарушение.

Правило 4. `CASCADE_NO_PARENT`

Поле	Значение
Название	Цель не связана с вышестоящей
Объект проверки	цель (`GOAL`)
Важность	`major`
Условие нарушения	Активная цель принадлежит подразделению, у которого `parent_id` не пуст, и у цели `parent_goal_id` пуст
`issue.object_id`	`goal.id`
`issue.details`	`Не указана вышестоящая цель`

```
SELECT g.id AS object_id, g.unit_id
FROM active_goal g
JOIN unit u ON u.id = g.unit_id
WHERE u.parent_id IS NOT NULL
  AND g.parent_goal_id IS NULL;
```

- Цели подразделений на вершине иерархии (`parent_id` пуст) вышестоящую цель иметь не обязаны, правило их не проверяет.
- Статус вышестоящей цели не проверяется: если родитель закрыт, нарушения нет. Расширение вынесено за рамки первой версии.

Справочник правил (данные таблицы `rule`)

code	title	object_type	severity	recommendation
`GOAL_NO_OWNER`	Цель без ответственного	`GOAL`	`critical`	Назначьте в карточке цели ответственного сотрудника. Если неясно, кто он, определите его вместе с руководителем подразделения
`GOAL_NO_KR`	У цели нет ключевых результатов	`GOAL`	`critical`	Добавьте к цели хотя бы один КР и сразу укажите целевое значение и вес. Если цель по смыслу нельзя измерить, пересмотрите её формулировку вместе с руководителем
`KR_WEIGHTS_SUM`	Веса КР не дают 100%	`GOAL`	`major`	Перераспределите веса КР так, чтобы сумма составила ровно 100%. Если КР один, его вес должен быть 100%
`CASCADE_NO_PARENT`	Цель не связана с вышестоящей	`GOAL`	`major`	Выберите в карточке цели вышестоящую цель, которую она поддерживает. Если подходящей нет, обсудите с руководителем, нужна ли эта цель

Как правила работают вместе

- Одна цель может нарушать несколько правил, тогда создаётся по одной проблеме на каждое нарушенное правило. Пара `(rule_code, object_id)` уникальна в прогоне.
- «Цель с проблемами» в `run_unit_stat.goals_with_issues` считается один раз, даже если у неё три проблемы: `COUNT(DISTINCT object_id)`.
- Значение `issue.unit_id` берётся из `goal.unit_id` на момент прогона.

Граничные данные для тестов

Данные	Результат
`owner_id` из пробелов	Нарушение `GOAL_NO_OWNER`
Цель со сроком в день снимка	Проверяется
Цель со сроком на день раньше снимка	Не проверяется
Веса 50 + 50	Нарушения нет
Веса 33,33 + 33,33 + 33,33	Нарушение `KR_WEIGHTS_SUM`, сумма 99,99
Единственный КР с весом 100	Нарушения нет
Цель без КР	Только `GOAL_NO_KR`
Цель в подразделении на вершине иерархии без `parent_goal_id`	Нарушения `CASCADE_NO_PARENT` нет

3. Метрики качества данных

Три метрики считаются по результатам успешного прогона и охвату выбранного подразделения.

Общие определения

- **Охват подразделения** — само подразделение и все его дочерние подразделения на любом уровне вложенности. Иерархия берётся из `unit.parent_id` на момент запроса и одинакова для текущего и прошлого прогона.
- **Прогон `R`** — выбранный прогон со статусом `SUCCESS`. Для прогонов `RUNNING` и `FAILED` метрик нет.

- **Прошлый прогон `R_prev`** — прогон из `check_run.compared_to_run_id` для `R` (выбор описан в разделе 4).

```
WITH RECURSIVE scope AS (
  SELECT id FROM unit WHERE id = :unit_id
  UNION ALL
  SELECT u.id FROM unit u JOIN scope s ON u.parent_id = s.id
)
```

Метрики

Ключ	Название	Единица	`higher_is_better`	Что лучше
`clean_goals_share`	Доля корректных целей	%	`true`	рост
`issues_total`	Число найденных проблем	шт.	`false`	снижение
`resolved_share`	Доля исправленных проблем	%	`true`	рост

Метрика 1. Доля корректных целей (`clean_goals_share`)

```
value = 100 × (Σ active_goals − Σ goals_with_issues) ÷ Σ active_goals
```

Суммы берутся по строкам `run_unit_stat` с `run_id = R` и `unit_id` из охвата.

```
SELECT CASE WHEN COALESCE(SUM(active_goals), 0) = 0 THEN NULL
            ELSE ROUND(100.0 * (SUM(active_goals) - SUM(goals_with_issues))
                       / SUM(active_goals), 2) END AS value
FROM run_unit_stat
WHERE run_id = :run_id AND unit_id IN (SELECT id FROM scope);
```

Если в охвате нет активных целей, значение `null`, а не 0% и не 100%.

Метрика 2. Число найденных проблем (`issues_total`)

```
SELECT COUNT(*) AS value
FROM issue
WHERE run_id = :run_id AND unit_id IN (SELECT id FROM scope);
```

Считается каждое нарушение: цель с двумя нарушениями даёт две проблемы. Разбивка по важности — тот же запрос с `GROUP BY severity`, в ответе всегда все три уровня (`critical`, `major`, `minor`), с нулём там, где проблем нет.

Если прогон успешен и проблем нет, значение равно 0. Это не «нет данных»: проверка прошла, и проблем действительно нет.

Метрика 3. Доля исправленных проблем (`resolved_share`)

Пусть `prev_set` — ключи `(rule_code, object_id)` проблем прошлого прогона в охвате, а `cur_set` — то же для текущего.

```
value = 100 × |prev_set − cur_set| ÷ |prev_set|
```

```
WITH prev AS (
  SELECT rule_code, object_id FROM issue
  WHERE run_id = :prev_run_id AND unit_id IN (SELECT id FROM scope)
), cur AS (
  SELECT rule_code, object_id FROM issue
  WHERE run_id = :run_id AND unit_id IN (SELECT id FROM scope)
)
SELECT CASE WHEN (SELECT COUNT(*) FROM prev) = 0 THEN NULL
            ELSE ROUND(100.0 * (SELECT COUNT(*) FROM (SELECT * FROM prev
EXCEPT SELECT * FROM cur) x)
                       / (SELECT COUNT(*) FROM prev), 2) END AS value;
```

Значение `null`, если прошлого прогона нет или в нём в этом охвате не было проблем (делить не на что).

Прошлое значение `resolved_share` — та же метрика, рассчитанная для прогона `R_prev` относительно его собственного прошлого прогона. Оно `null`, если у `R_prev` нет прошлого прогона или в его охвате в прошлом прогоне не было проблем. Тогда `delta` и `trend` тоже `null`.

Подразделение вместе с дочерними

Суммируются количества, и только потом делится. Доли дочерних подразделений не усредняются.

Пример на числах сценария 1 документа бизнес-аналитика: «Департамент А» (6 активных целей, 2 с проблемами) и дочернее «Управление Б» (4 активные цели, 2 с проблемами).

- Правильно: (6 + 4 − 2 − 2) ÷ (6 + 4) = 6 ÷ 10 = 60,00%.
- Неправильно (среднее долей): (66,67% + 50,00%) ÷ 2 = 58,33%. Малое подразделение влияло бы на результат наравне с большим.

Округление, разница и оценка

- Проценты округляются до двух знаков после запятой (`ROUND(x, 2)`, математическое округление). Счётчики целые.
- **Разница** `delta` = `value` − `previous` по уже округлённым значениям: 60,00 − 70,00 = −10,00.
- **Оценка** `trend` зависит от `higher_is_better`, а не от знака разницы:

Условие	`trend`
`value` или `previous` равно `null`	`null`
`delta = 0`	`same`
`delta > 0` и `higher_is_better = true`, либо `delta < 0` и `higher_is_better = false`	`better`
иначе	`worse`

Значения в пустых случаях

Ситуация	`clean_goals_share`	`issues_total`	`resolved_share`
Первый прогон	значение есть, `previous` и `delta` равны `null`	значение есть, `previous` и `delta` равны `null`	`null`
Охват без активных целей	`value` равно `null`; `previous` может быть числом, тогда `delta` и `trend` равны `null`	0	`null`, если в прошлом прогоне в охвате не было проблем
Успешный прогон, проблем нет	100,00	0	100,00, если проблемы прошлого прогона все исправлены

Ситуация	`clean_goals_share`	`issues_total`	`resolved_share`
В прошлом прогоне проблем не было	значение есть	значение есть	`null`

Общее правило: метрики, которые невозможно рассчитать, равны `null`. Счётчики проблем могут быть 0: это значение, а не отсутствие данных.

4. Дельта с прошлым прогоном

Дельта показывает, какие проблемы появились, остались и исправлены с прошлого прогона, и как изменились метрики. Сравнивается только ближайший прошлый успешный прогон.

Когда проблема «та же самая»

Ключ проблемы — пара `(rule_code, object_id)`. Две проблемы из разных прогонов одинаковы, если совпадают оба значения.

Поле	Входит в ключ	Почему
`rule_code`, `object_id`	да	Определяют, какое правило нарушено в какой цели
`severity`	нет	Важность правила могут изменить. Проблема остаётся той же, показывается текущая важность
`details`	нет	Текст меняется: сумма весов была 80%, стала 90%, а правило и цель те же
`unit_id`	нет	Цель могла сменить подразделение, а проблема в ней нет
`issue.id`	нет	Идентификатор записи в конкретном прогоне, у каждого прогона свой

Три категории

Пусть `cur` — ключи проблем текущего прогона в охвате выбранного подразделения, `prev` — то же для прошлого прогона.

`delta_status`	Определение	Множество
`new`	Проблема есть сейчас и не была найдена в прошлом прогоне	`cur − prev`
`persisting`	Проблема есть и сейчас, и была в прошлом прогоне	`cur ∩ prev`
`resolved`	Проблема была в прошлом прогоне и сейчас не найдена	`prev − cur`

```
-- new
SELECT rule_code, object_id FROM cur EXCEPT SELECT rule_code, object_id FROM
prev;
-- persisting
SELECT rule_code, object_id FROM cur INTERSECT SELECT rule_code, object_id
FROM prev;
-- resolved
SELECT rule_code, object_id FROM prev EXCEPT SELECT rule_code, object_id FROM
cur;
```

- `new + persisting` равно числу проблем текущего прогона; `persisting + resolved` равно числу проблем прошлого прогона.
- Исправленная проблема не хранится в `issue` текущего прогона. Её описание и важность берутся из записи `issue` прошлого прогона.
- Если проблему исправили, а потом она появилась снова, она считается `new`: сравнение идёт только с ближайшим прошлым прогоном.
- Если цель перенесли в другое подразделение, в охвате прежнего подразделения проблема будет `resolved`, а в охвате нового `new`. Первая версия принимает это как допущение.

Какой прогон считается прошлым

Прошлый прогон определяется один раз, когда прогон завершается успешно, и сохраняется в `check_run.compared_to_run_id`. Дальше он не пересчитывается.

> **Прошлый прогон для `R`** — прогон со статусом `SUCCESS` и наибольшим `started_at` среди прогонов с `started_at` меньше, чем у `R`. Если такого прогона нет, `compared_to_run_id` пуст.

```
SELECT id FROM check_run
WHERE status = 'SUCCESS' AND started_at < :r_started_at
ORDER BY started_at DESC
LIMIT 1;
```

Одновременно выполняется не более одного прогона: новый не запускается, пока есть прогон в статусе `RUNNING`.

Если прошлый прогон завершился с ошибкой

Прогоны `FAILED` и `RUNNING` пропускаются при выборе прошлого прогона: сравнение идёт с ближайшим предшествующим успешным.

Прогон	Статус	С чем сравнивается
RUN-005	`SUCCESS`	RUN-004
RUN-006	`FAILED`	не сравнивается, результаты не отдаются
RUN-007	`SUCCESS`	RUN-005 (неудачный RUN-006 пропущен)

- В ответе API поле `compared_to` показывает RUN-005 и его дату, поэтому видно, что сравнение идёт не с предыдущим по календарю, а с последним успешным.
- У неуспешного прогона дельты и метрик нет: API возвращает 409 `RUN_FAILED` (раздел 5).
- Если успешных прогонов раньше не было, действует правило первого прогона.

Первый прогон

Первый прогон — успешный прогон без предшествующего успешного (`compared_to_run_id` пуст).

Что	Значение
`compared_to`	`null`
`previous`, `delta`, `trend` у каждой метрики	`null`, не 0
`previous`, `delta` в разбивке по важности	`null`
Блок `delta` (новые, сохраняющиеся, исправленные)	`null` целиком

Что	Значение
`delta_status`, `runs_streak` у проблемы	`null`
Интерфейс	«нет данных для сравнения»

Различие между `null` и 0 принципиально: если сравнение есть, а исправленных проблем нет, в блоке дельты стоит `resolved: 0`. Если сравнения нет, блока нет вовсе, потому что «0 исправленных» означало бы «сравнили и не нашли», а это неправда.

История проблемы по прогонам

`runs_streak` — число прогонов подряд, считая текущий, в которых присутствует проблема с тем же ключом. Цепочка идёт назад по `compared_to_run_id`. У проблемы `new` значение равно 1, у `resolved` и при отсутствии сравнения равно `null`. Значение вычисляется при запросе и не хранится.

Граничные случаи

Ситуация	Результат
Единственный успешный прогон	Первый прогон: все сравнения `null`
В прошлом прогоне проблем не было	Сравнение есть: `new` — все текущие, `persisting` и `resolved` равны 0. `resolved_share` равна `null`
Проблема исправлена и вернулась через два прогона	`new`
Изменилась `severity` правила между прогонами	Остаётся `persisting`, показывается текущая важность
Прошлый прогон `RUNNING`	Пропускается, как `FAILED`

5. Контракт API

Три метода обслуживают страницу ККД: список прогонов, сводка по подразделению и список проблем. Все числа отдаются как есть, фронтенд ничего не пересчитывает. Поля, которые могут быть пустыми, всегда присутствуют в ответе со значением `null`.

Общие правила

- `run_id` и `unit_id` обязательны в сводке и в списке проблем: у них нет значений по умолчанию.
- Формат ошибки единый: `{"error": {"code": "...", "message": "..."}}`.
- Порядок проверки при нескольких ошибках: параметры (422), существование прогона и подразделения (404), состояние прогона (409).

HTTP	`code`	Когда
404	`RUN_NOT_FOUND`	Прогона с таким `run_id` нет
404	`UNIT_NOT_FOUND`	Подразделения с таким `unit_id` нет
409	`RUN_FAILED`	Прогон завершился с ошибкой, результатов нет
409	`RUN_IN_PROGRESS`	Прогон ещё выполняется, результатов пока нет
422	`INVALID_PARAMETER`	Параметр отсутствует, имеет неверный тип или недопустимое значение. В `message` указано, какой именно и почему

```
{"error": {"code": "RUN_NOT_FOUND", "message": "Прогон RUN-999 не найден"}}
```

Метод 1. `GET /api/runs` — список прогонов

Параметров нет. Сортировка: по `snapshot_date` от новых к старым. Ошибок нет: пустая система возвращает `{"items": []}`.

Поле ответа	Тип	Описание
`items[].id`	string	Идентификатор прогона, например `RUN-008`
`items[].snapshot_date`	date	Дата снимка данных
`items[].status`	string	`RUNNING`, `SUCCESS`, `FAILED`
`items[].compared_to`	object или `null`	`{id, snapshot_date}` прошлого прогона. `null` у первого прогона и у неуспешных
`items[].issues_count`	integer или `null`	Число проблем по всей организации. `null`, если статус не `SUCCESS`

Метод 2. `GET /api/kkd/summary` — сводка по подразделению

Параметр	Тип	Обязательный	Описание
`run_id`	string	да	Идентификатор прогона
`unit_id`	string	да	Подразделение. Расчёт идёт по его охвату (само и все дочерние)

Ошибки: 422 (нет `run_id` или `unit_id`), 404 `RUN_NOT_FOUND`, 404 `UNIT_NOT_FOUND`, 409 `RUN_FAILED`, 409 `RUN_IN_PROGRESS`.

Поле ответа	Тип	Описание
`run`	object	`{id, snapshot_date}` выбранного прогона
`compared_to`	object или `null`	`{id, snapshot_date}` прошлого прогона. `null` у первого прогона
`unit`	object	`{id, name}` выбранного подразделения
`metrics[]`	array	Всегда три метрики в порядке `clean_goals_share`, `issues_total`, `resolved_share`
`metrics[].key`, `title`, `unit`	string	Ключ, название, единица измерения (`%` или `шт.`)
`metrics[].higher_is_better`	boolean	`true`, если рост — это хорошо
`metrics[].value`	number или `null`	Текущее значение
`metrics[].previous`	number или `null`	Значение в прошлом прогоне
`metrics[].delta`	number или `null`	`value − previous` по округлённым значениям
`metrics[].trend`	string или `null`	`better`, `worse`, `same` или `null` (нет сравнения)
`issues_by_severity[]`	array	Всегда три элемента: `critical`, `major`, `minor`, в этом порядке

Поле ответа	Тип	Описание
`issues_by_severity[].code`	string	Уровень важности
`issues_by_severity[].current`, `previous`, `delta`	integer или `null`	Число проблем сейчас, в прошлом прогоне и разница. `previous` и `delta` равны `null` при отсутствии сравнения
`delta`	object или `null`	`{new, persisting, resolved}`. `null` у первого прогона

Проценты имеют до двух знаков после запятой, счётчики целые.

Полный пример ответа. Числа взяты из сценария 1 документа бизнес-аналитика: «Департамент А» вместе с «Управлением Б».

Запрос: `GET /api/kkd/summary?run_id=RUN-008&unit_id=DEP-A`


```
{
  "run": { "id": "RUN-008", "snapshot_date": "2026-10-26" },
  "compared_to": { "id": "RUN-007", "snapshot_date": "2026-10-19" },
  "unit": { "id": "DEP-A", "name": "Департамент А" },
  "metrics": [
    {
      "key": "clean_goals_share",
      "title": "Доля корректных целей",
      "unit": "%",
      "higher_is_better": true,
      "value": 60.00,
      "previous": 70.00,
      "delta": -10.00,
      "trend": "worse"
    },
    {
      "key": "issues_total",
      "title": "Число найденных проблем",
      "unit": "шт.",
      "higher_is_better": false,
      "value": 7,
      "previous": 5,
      "delta": 2,
      "trend": "worse"
    },
    {
      "key": "resolved_share",
      "title": "Доля исправленных проблем",
```

```
      "unit": "%",
      "higher_is_better": true,
      "value": 60.00,
      "previous": 40.00,
      "delta": 20.00,
      "trend": "better"
    }
  ],
  "issues_by_severity": [
    { "code": "critical", "current": 2, "previous": 1, "delta": 1 },
    { "code": "major", "current": 5, "previous": 4, "delta": 1 },
    { "code": "minor", "current": 0, "previous": 0, "delta": 0 }
  ],
  "delta": { "new": 5, "persisting": 2, "resolved": 3 }
}
```

Проверка числами: 2 + 5 + 0 = 7 проблем; 5 новых + 2 сохраняющиеся = 7; 2 сохраняющиеся + 3 исправленные = 5 проблем прошлого прогона; 3 ÷ 5 = 60,00%.

Метод 3. `GET /api/kkd/issues` — список проблем

Параметр	Тип	Обязательный	Описание
`run_id`	string	да	Идентификатор прогона
`unit_id`	string	да	Подразделение (охват: само и дочерние)
`rule_code`	string	нет	Фильтр по правилу. Значение должно быть в справочнике правил
`severity`	string	нет	`critical`, `major` или `minor`
`delta_status`	string	нет	`new`, `persisting` или `resolved`
`limit`	integer	нет	Размер страницы от 1 до 100. По умолчанию 20
`offset`	integer	нет	Сдвиг, 0 или больше. По умолчанию 0

Фильтры комбинируются через «и». Сортировка фиксирована: сначала по важности (`critical`, `major`, `minor`), затем по `rule.code`, затем по `object.id`. Постоянный порядок гарантирует, что страницы не перемешиваются.

Что входит в список. Без параметра `delta_status` список содержит только проблемы выбранного прогона (`new` и `persisting`): `total` без фильтров равно значению метрики `issues_total` в сводке. Исправленные проблемы

показываются только при явном `delta_status=resolved`, фильтры `rule_code` и `severity` применяются и к ним.

Ошибки: 422 (нет `run_id` или `unit_id`; `severity`, `delta_status` или `rule_code` с недопустимым значением; `limit` вне диапазона 1–100; `offset` меньше 0 или не число), 404 `RUN_NOT_FOUND`, 404 `UNIT_NOT_FOUND`, 409 `RUN_FAILED`, 409 `RUN_IN_PROGRESS`.

Поле ответа	Тип	Описание
`items[]`	array	Проблемы текущей страницы
`items[].id`	string	Идентификатор записи проблемы
`items[].run_id`	string	Прогон, в котором проблема записана. Для `resolved` это прошлый прогон
`items[].rule`	object	`{code, title}`
`items[].severity`	string	Важность. У `resolved` — на момент прошлого прогона
`items[].delta_status`	string или `null`	`new`, `persisting`, `resolved`. `null`, если сравнения нет
`items[].unit`	object	`{id, name}` подразделения цели
`items[].object`	object	`{type, id, title}`: тип `GOAL`, идентификатор и название цели
`items[].details`	string	Что именно не так
`items[].recommendation`	string	Как исправить
`items[].runs_streak`	integer или `null`	Сколько прогонов подряд (с текущим) присутствует эта проблема. `null` у `resolved` и при отсутствии сравнения
`total`	integer	Число проблем, подходящих под фильтры, без учёта страницы
`limit`, `offset`	integer	Применённые значения

Пример одного элемента:

```
{
  "id": "ISS-0412",
  "run_id": "RUN-008",
  "rule": { "code": "KR_WEIGHTS_SUM", "title": "Веса КР не дают 100%" },
  "severity": "major",
  "delta_status": "persisting",
  "unit": { "id": "DEP-A", "name": "Департамент А" },
  "object": { "type": "GOAL", "id": "G-117", "title": "Увеличить число
зарплатных клиентов" },
  "details": "Сумма весов КР: 80.00%, ожидается 100%",
  "recommendation": "Перераспределите веса КР так, чтобы сумма составила
ровно 100%. Если КР один, его вес должен быть 100%",
  "runs_streak": 3
}
```

Граничные случаи

Общее правило для пустых случаев: метрики, которые невозможно рассчитать, равны `null`; счётчики проблем могут быть 0.

Ситуация	Что отвечает API
Подразделение без целей (в охвате нет активных целей)	Сводка: 200. `clean_goals_share`: `value` равно `null`, `delta` и `trend` равны `null`, `previous` может быть числом; `issues_total`: `value` 0; `resolved_share`: `null`, если в прошлом прогоне в охвате не было проблем; все три строки `issues_by_severity` с нулями; `delta` равно `{new: 0, persisting: 0, resolved: 0}`, если сравнение есть. Список проблем: 200, `items: []`, `total: 0`. Это не ошибка
Цель без ключевых результатов	Правило `GOAL_NO_KR` создаёт одну проблему, `KR_WEIGHTS_SUM` к цели не применяется. Цель считается активной с проблемами и входит в знаменатель доли корректных целей
Прогон ещё выполняется	Сводка и список проблем: 409 `RUN_IN_PROGRESS`. В списке прогонов виден со статусом `RUNNING`, `compared_to` и `issues_count` равны `null`

Ситуация	Что отвечает API
Прогон завершился с ошибкой	Сводка и список проблем: 409 `RUN_FAILED`. В списке прогонов статус `FAILED`, `compared_to` и `issues_count` равны `null`
Первый прогон	Сводка: `compared_to` и `delta` равны `null`; у каждой метрики `previous`, `delta`, `trend` равны `null`; в `issues_by_severity` `previous` и `delta` равны `null`. Список проблем: `delta_status` и `runs_streak` равны `null`
Фильтр `delta_status` при отсутствии сравнения	200, `items: []`, `total: 0`. Отличить «нет сравнения» от «нет проблем» можно по сводке: `compared_to` равно `null`
Фильтры не дают результатов	200, `items: []`, `total: 0`
`offset` больше `total`	200, `items: []`, `total` равно реальному числу

Допущения

Заказчика опросить нельзя, поэтому ниже собрано всё, что принято без его ответа. Каждое допущение можно подтвердить или заменить без пересмотра остальной спецификации.

№	Допущение	Где применяется
1	Активная цель — статус `ACTIVE` и срок `deadline` не раньше даты снимка	Раздел 2
2	Иерархия подразделений берётся на момент запроса и одинакова для текущего и прошлого прогона. Переносы подразделений между прогонами не учитываются	Раздел 3
3	Если цель перенесли в другое подразделение, в охвате прежнего подразделения проблема считается исправленной, в охвате нового новой	Раздел 4

№	Допущение	Где применяется
4	Цели подразделений на вершине иерархии вышестоящую цель иметь не обязаны	Раздел 2, правило `CASCADE_NO_PARENT`
5	Статус вышестоящей цели не проверяется	Раздел 2, правило `CASCADE_NO_PARENT`
6	Ответственный хранится как внешний идентификатор. Заблокированные или уволенные сотрудники не проверяются	Раздел 1, правило `GOAL_NO_OWNER`
7	Вес KP обязателен, от 0 до 100. Сумма весов сравнивается с 100 точно, без допуска	Раздел 1, правило `KR_WEIGHTS_SUM`
8	Одновременно выполняется не более одного прогона	Раздел 4
9	Ролевой доступ в спецификации не описан: какие роли видят какие подразделения, уточняется отдельно (вопрос 7 документа бизнес-аналитика)	Раздел 5

Мотивация участия в проекте

Почему мне интересен этот проект

Проект интересен тем, что это реальная задача: нужно превратить требования бизнеса в спецификацию, по которой команда реализует модуль без возвратов с вопросами, то есть описать модель данных, правила проверки, метрики и контракт API. Пока я делал тестовое задание, меня особенно зацепила однозначность формулировок: если условие можно прочитать двумя способами, разработчик реализует его по-своему. Я хочу попробовать свои силы в таком проекте и стажироваться в крупной компании, чтобы учиться на реальных процессах и получать обратную связь от команды.

Как я вижу свою роль в команде

Я вижу себя системным аналитиком, который переводит требования заказчика и бизнес-аналитика в техническую спецификацию для разработки. В мою работу входит общение с заказчиком и сбор требований, их формализация, подготовка документации: функционально-технических требований,

технических заданий для команды разработки, описания API и инструкций для пользователей. Я также готов участвовать в тестировании, сверяя реализованное со спецификацией. Мне близка роль человека, который держит общую коммуникацию в команде, и в перспективе я хотел бы развиваться в сторону координации работы команды.

Сколько времени я готов уделять проекту

Я готов уделять проекту от 20 часов в неделю в течение 3 месяцев.